

P0330R2

Literal Suffixes for ptrdiff_t and size_t

Published Proposal, 2018-10-01

Authors:

[JeanHeyd Meneide](#)
Rein Halbersma

Audience:

EWG, CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/ThePhD/future_cxx/blob/master/papers/source/d0330.bs

Current:

https://rawgit.com/ThePhD/future_cxx/master/papers/d0330.html

Abstract

This paper proposes core language suffixes for size_t and ptrdiff_t.

Table of Contents

1	Revision History
1.1	Revision 0 - November 21st, 2014
1.2	Revision 1 - October 12th, 2017
1.3	Revision 2 - October 1st, 2018
2	Motivation
3	Design
3.1	What about the fixed/least/max (unsigned) int types?
4	Impact on the Standard
5	Proposed wording and Feature Test Macros
5.1	Proposed feature Test Macro
5.2	Intent
5.3	Proposed Wording
6	Acknowledgements
7	Feedback on Revisions
	References
	Informative References

§ 1. Revision History

§ 1.1. Revision 0 - November 21st, 2014

Initial release; published as N4254.

Published as P0330R0; summarized LEWG's view re N4254; dropped the proposed suffix for ptrdiff_t; changed the proposed suffix for size_t to zu; added survey of existing literal suffixes.

§ 1.2. Revision 1 - October 12th, 2017

Published as P0330R1; expanded the survey of existing literals. Synced the proposed wording with the Working Draft WG21/N4687. Moved the reference implementation from BitBucket to GitHub.

§ 1.3. Revision 2 - October 1st, 2018

Published as P0330R2; change reply-to and point of contact from Rein Halbersma to JeanHeyd Meneide, who revitalized paper according to feedback from Rein Halbersma, and all of LWG. Overwhelming consensus for it to be a Language Feature instead, proposal rewritten as Language proposal with wording against [\[n4762\]](#).

§ 2. Motivation

Consider this very simple code to print an index and its value:

```
std::vector<int> v{0, 1, 2, 3};
for (auto i = 0; i < v.size(); ++i) {
    std::cout << i << ": " << v[i] << '\n';
}
```

This code can lead to the following warnings:

```
main.cpp: In function 'int main()':
main.cpp:warning: comparison of integer expressions
of different signedness: 'int' and 'long unsigned int' [-Wsign-compare]
    for (auto i = 0; i < v.size(); ++i) {
                        ~~~~~
```

It grows worse if a user wants to cache the size rather than query it per-iteration:

```
std::vector<int> v{0, 1, 2, 3};
for (auto i = 0, s = v.size(); i < s; ++i) {
    /* use both i and v[i] */
}
```

Resulting in a hard compiler error:

```
main.cpp: In function 'int main()':
main.cpp:8:10: error: inconsistent deduction
for 'auto': 'int' and then 'long unsigned int'
    for (auto i = 0, s = v.size(); i < s; ++i) {
          ^~~~~
```

This paper proposes adding a zu literal suffix that deduces literals to size_t, making the following warning-free:

```
for (auto i = 0zu; i < v.size(); ++i) {
    std::cout << i << ": " << v[i] << '\n';
}
```

More generally:

- int is the default type deduced from integer literals without suffix;

- comparisons, signs or conversion ranks with integers can lead to surprising results;
- `size_t` -- and less frequently, `ptrdiff_t` -- are nearly impossible to avoid in the standard library for element access or `.size()` members;
- programmer intent and stability is hard to communicate portably with the current set of literals;
- surprises range from (pedantic) compiler errors to undefined behavior;
- existing integer suffixes (such as `ul`) are not a general solution, e.g. when switching compilation between 32-bit and 64-bit on common architectures;
- and, C-style casts and `static_casts` are verbose.

§ 3. Design

Following the feedback from [§7 Feedback on Revisions](#), we have dropped the `std::support_literals` User-Defined Literals and chose a Core Language Literal Suffix. We opine that it would better serve the needs of addressing the motivation.

As a language feature, the design of the suffixes becomes much simpler. The core language only has one format for its integer literal suffixes: the letter(s), with an optional `u` on either side of the letter(s) to make it unsigned, with the signed variant being the default on most architectures. This ruled out using `s` and `sz`, because that would produce an inconsistent set of suffixes with the rest of the language.

The literal suffixes `z` and `uz/zu` were chosen to represent signed and unsigned, respectively. `decltype(0z)` will yield `ptrdiff_t` and `decltype(0uz)/decltype(0zu)` will yield `size_t`. Like other case-insensitive language literal suffixes, it will accept both `Z` and `z` (and `U` and `u` alongside of it). This follows the current convention of the core language to be able to place `u` and `z` in any order / any case for the suffix.

§ 3.1. What about the fixed/least/max (unsigned) int types?

This paper does not propose suffixes for the fixed size, at-least size, and max size integral types in the standard library or the language. This paper is focusing exclusively on `ptrdiff_t` and `size_t`. We have also been made aware of another paper which may handle this separately and considers all the design space necessary for such.

§ 4. Impact on the Standard

This feature is purely an extension of the language and has, to the best of our knowledge, no conflict with existing or currently proposed features. `z` is currently not a literal suffix in the language.

§ 5. Proposed wording and Feature Test Macros

The following wording is relative to [\[n4762\]](#).

§ 5.1. Proposed feature Test Macro

The recommended feature test macro is `__cpp_ptrdiff_t_suffix`.

§ 5.2. Intent

The intent of this paper is to propose 2 language suffixes for integral literals of specific types. One is for `ptrdiff_t`, one is for `size_t`. We follow the conventions set out for other literals in the standard. We define the suffix to produce types `size_t` and `ptrdiff_t` similar to how [§5.13.7 Pointer Literals](#) [\[lex.nullptr\]](#) introduces `std::nullptr_t`.

§ 5.3. Proposed Wording

Modify §5.13.2 Integer Literals [**lex.icon**] with additional suffixes:

integer-suffix:

unsigned-suffix long-suffix_{opt}

unsigned-suffix long-long-suffix_{opt}

unsigned-suffix ptrdiff-suffix_{opt}

long-suffix unsigned-suffix_{opt}

long-long-suffix unsigned-suffix_{opt}

ptrdiff-suffix unsigned-suffix_{opt}

unsigned-suffix: one of

u U

long-suffix: one of

l L

long-long-suffix: one of

ll LL

ptrdiff-suffix: one of

z Z

Append to §5.13.2 Integer Literals [**lex.icon**]'s **Table 7** two additional entries:

Suffix	Decimal literal	Binary, octal, or hexadecimal literal
z or Z	ptrdiff t	ptrdiff t
Both u or U and z or Z	size t	size t

Append to §14.8.1 Predefined macro names [**cpp.predefined**]'s **Table 16** with one additional entry:

Macro name	Value
_cpp_ptrdiff_t_suffix	201811L

§ 6. Acknowledgements

Thank you to Rein Halbersma, who started this paper and put in the necessary work for r0 and r1. Thank you to Walter E. Brown, who acted as *locum* on this paper before the Committee twice and gave us valuable feedback on wording. Thank you to Lounge<C++>'s Cicada for encouraging us to write this paper. Thank you to Hubert Tong for giving us a few pointers on where in the Core Language to modify things for such a paper. Thank you to Tim Song for wording advice.

We appreciate your guidance as we learn to be a better Committee member and represent the C++ community's needs more more efficiently and effectively in the coming months.

§ 7. Feedback on Revisions

Polls are in the form **Strongly in Favor** | **Favor** | **Neutral** | **Against** | **Strongly Against**. The polls on Revision 1 were as follows, from LWG at the WG21 Albuquerque meeting.

Proposal as presented, i.e., are we OK with the library solution going forward?

0 | 6 | 5 | 7 | 4

We translated this as strong discouragement to pursue this feature as a set of user-defined literals. A second poll was taken.

Do we want to solve this problem with a language feature?

2 | 15 | 0 | 2 | 2

We considered this overwhelming consensus for it to be a language feature instead, culminating in this paper after much feedback.

§ References

§ Informative References

[N4762]

ISO/IEC JTC1/SC22/WG21 - The C++ Standards Committee; Richard Smith. [N4762 - Working Draft, Standard for Programming Language C++](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf). May 7th, 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/n4762.pdf>